Análise de Resiliência e Propagação de Vulnerabilidades em Grafos de Dependências do Ecossistema Python

Márcio Levi Sales Prado — RA 183680
MC859 — UNICAMP, Abril de 2026

1. Introdução

O ecossistema de pacotes Python (PyPI) é um dos maiores repositórios de software livre do mundo, com mais de 500 mil pacotes publicados. A cadeia de dependências entre esses pacotes forma uma rede complexa: quando um pacote amplamente usado contém uma vulnerabilidade de segurança, todos os projetos que dependem dele — direta ou indiretamente — ficam expostos ao mesmo risco.

Este projeto modela esse ecossistema como um **grafo direcionado**, no qual cada vértice representa um pacote e cada aresta **A** → **B** indica que A declara dependência direta de runtime sobre B. O objetivo central é identificar pacotes de alto risco combinando métricas estruturais do grafo (centralidade, grau, componentes fortemente conexas) com dados reais de uso (downloads mensais) e vulnerabilidades conhecidas do OSV (*Open Source Vulnerabilities*) e do GitHub Advisory Database.

Coleta de dados. A lista dos 3.000 pacotes mais baixados nos últimos 30 dias foi obtida do serviço público *top-pypi-packages* (hugovk.github.io), que agrega estatísticas oficiais do PyPI. Para cada pacote-semente, a API JSON do PyPI (pypi.org/pypi/{pkg}/json) foi consultada para extrair as dependências diretas de runtime declaradas no campo requires_dist, filtrando dependências opcionais (marcadas com extra ==). O processo expandiu o conjunto recursivamente via BFS até o limite de 8.000 vértices, resultando em **3.359 pacotes** e **9.659 arestas**. Todos os dados provêm de APIs públicas e não requerem anonimização. Além das dependências, cada nó foi anotado com o volume de downloads mensais proveniente da mesma fonte, utilizado para ponderar a análise de propagação de vulnerabilidades.

Repositório com instâncias e scripts: https://github.com/marcinholsp/Projeto_MC859

2. Tamanho do Grafo

Métrica	Valor
Número de vértices (pacotes)	3.359
Número de arestas (dependências)	9.659
Grau médio (in + out)	5.75
In-degree médio (dependentes diretos)	2.88
Out-degree médio (dependências diretas)	2.88
In-degree máximo	495 (typing-extensions)
Out-degree máximo	162
CFCs totais	3.351
Maior CFC	8 vértices
CFCs singleton (1 vértice)	3.349
Componentes fracamente conexas (CFrCs)	375
Maior componente fracamente conexa	2.948 vértices

A densidade do grafo é baixa (9.659 arestas para 3.359 nós), o que é esperado em redes de dependências reais: cada pacote declara, em média, menos de 3 dependências diretas. O in-degree máximo de 495 (typing-extensions) evidencia a presença de *hubs* — pacotes que servem de base para centenas de outros — característica típica de redes livres de escala (*scale-free*).

3. Distribuição de Graus

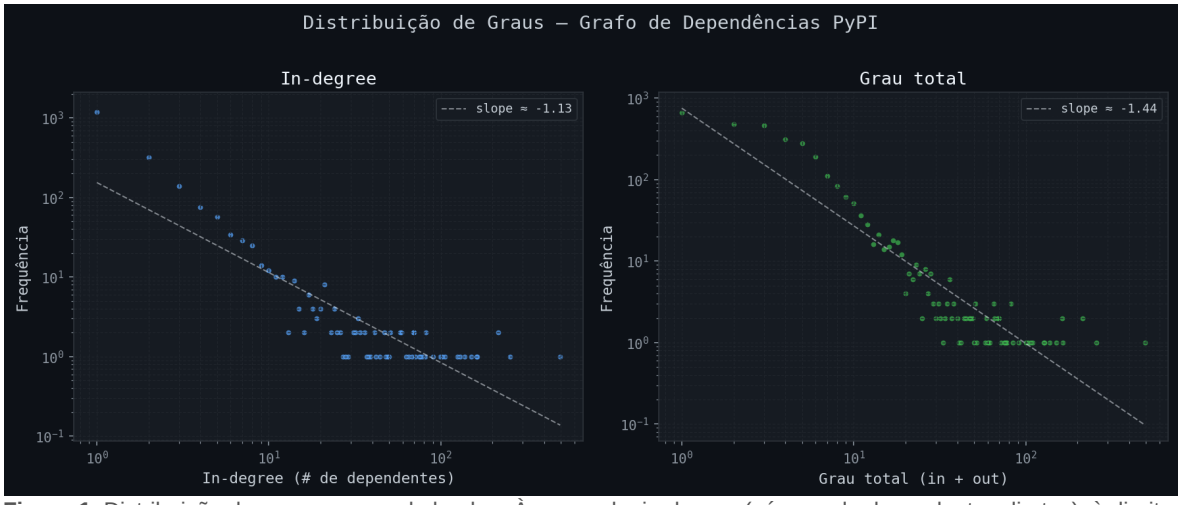

Figura 1. Distribuição de graus em escala log-log. À esquerda: in-degree (número de dependentes diretos); à direita: grau total (in + out). A linha tracejada mostra o ajuste por lei de potência.

A distribuição de graus segue uma **lei de potência** (*power-law*) em escala log-log, com expoentes estimados por regressão linear de –1,13 para o in-degree e –1,44 para o grau total. Esse padrão — característico de redes livres de escala — implica que a maioria dos pacotes tem poucos dependentes, enquanto um número muito reduzido de hubs concentra centenas deles. Do ponto de vista de segurança, esses hubs são candidatos naturais a pontos críticos: uma vulnerabilidade em typing-extensions (in-degree 495) ou requests (253) pode afetar, direta ou indiretamente, grande parte do ecossistema.

A Figura 2 apresenta os 20 pacotes com maior in-degree. Observa-se que typing-extensions destaca-se com folga — seu in-degree é quase o dobro do segundo colocado (requests). Também chamam atenção os pacotes pyobjc-core e pyobjc-framework-cocoa, específicos para macOS, que aparecem no top-10 estrutural mas têm downloads muito menores que numpy ou requests — ilustrando por que centralidade estrutural isolada não é suficiente para avaliar criticidade.

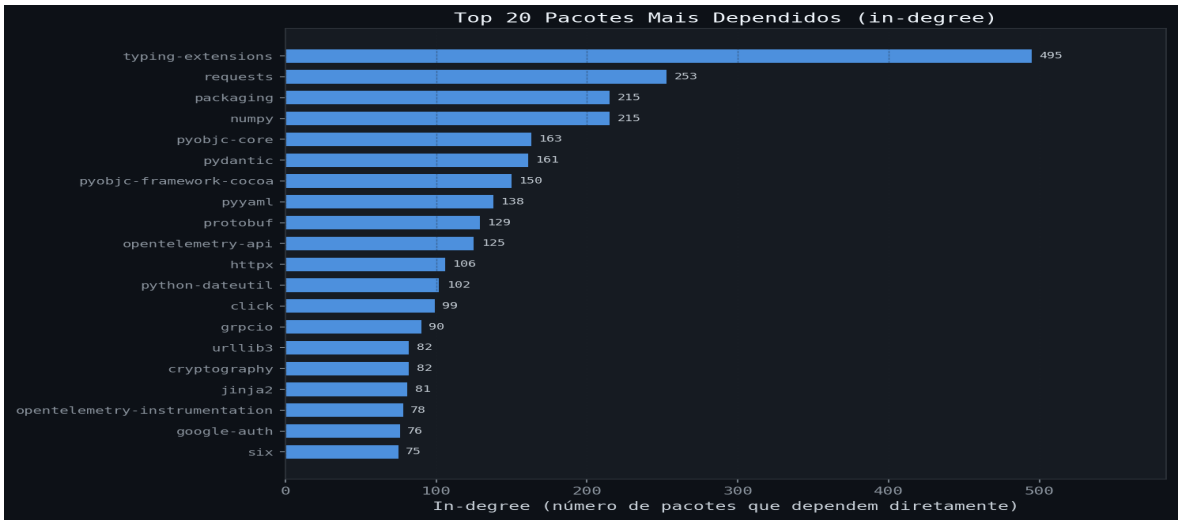

Figura 2. Top 20 pacotes por in-degree (número de pacotes que dependem diretamente de cada um).

4. Componentes Fortemente Conexas (CFCs)

O algoritmo de Kosaraju identificou **3.351 CFCs**. Desse total. **3.349 são singletons** (99.94%). confirmando que o grafo de dependências PyPI é essencialmente *acíclico*. Gerenciadores de pacotes como pip exigem um grafo acíclico para resolver dependências deterministicamente. de modo que dependências circulares são proibidas em teoria — as raras exceções encontradas (2 CFCs com mais de um vértice. de tamanhos 2 e **8**) podem indicar versões de compatibilidade cruzada ou opcionais não

declaradas corretamente.

A Figura 3 ilustra a distribuição de tamanhos das CFCs. O painel esquerdo evidencia o domínio absoluto dos singletons; o painel direito amplia as únicas duas CFCs com mais de um vértice, confirmando que dependências circulares são eventos raros e de pequena escala neste ecossistema.

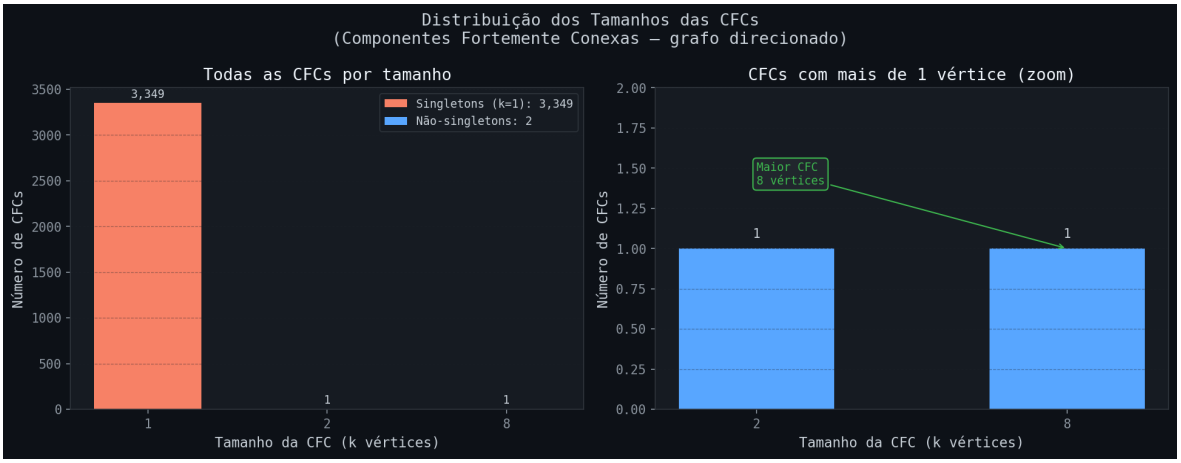

Figura 3. Distribuição dos tamanhos das CFCs. Esquerda: visão geral (3.349 singletons dominam). Direita: zoom nas 2 CFCs com mais de 1 vértice (tamanhos 2 e 8).

5. Componentes Fracamente Conexas (CFrCs)

Ignorando a direção das arestas. o grafo possui **375 componentes fracamente conexas**. A maior delas contém **2.948 vértices** (87.8% do total) — o chamado *componente gigante*. presente em praticamente todas as redes reais de larga escala. As demais 374 componentes são. em sua maioria. pacotes isolados ou pequenos subgrafos sem ligação com o núcleo principal do ecossistema. provavelmente pacotes de nicho ou com dependências não cobertas pelo escopo da coleta.

A existência de um componente gigante cobrindo quase 88% do grafo tem implicações diretas para a análise de propagação de vulnerabilidades: uma falha em qualquer pacote dentro desse componente pode, em princípio, atingir outros 2.947 pacotes por caminhos de dependência. Essa observação motiva o uso de modelos de propagação probabilística — como o Independent Cascade — em etapas futuras do projeto, para quantificar o alcance real considerando a intensidade de uso de cada dependência.

Instâncias disponíveis em: https://github.com/marcinholsp/Projeto_MC859 — formatos GraphML e GEXF, pasta data/.